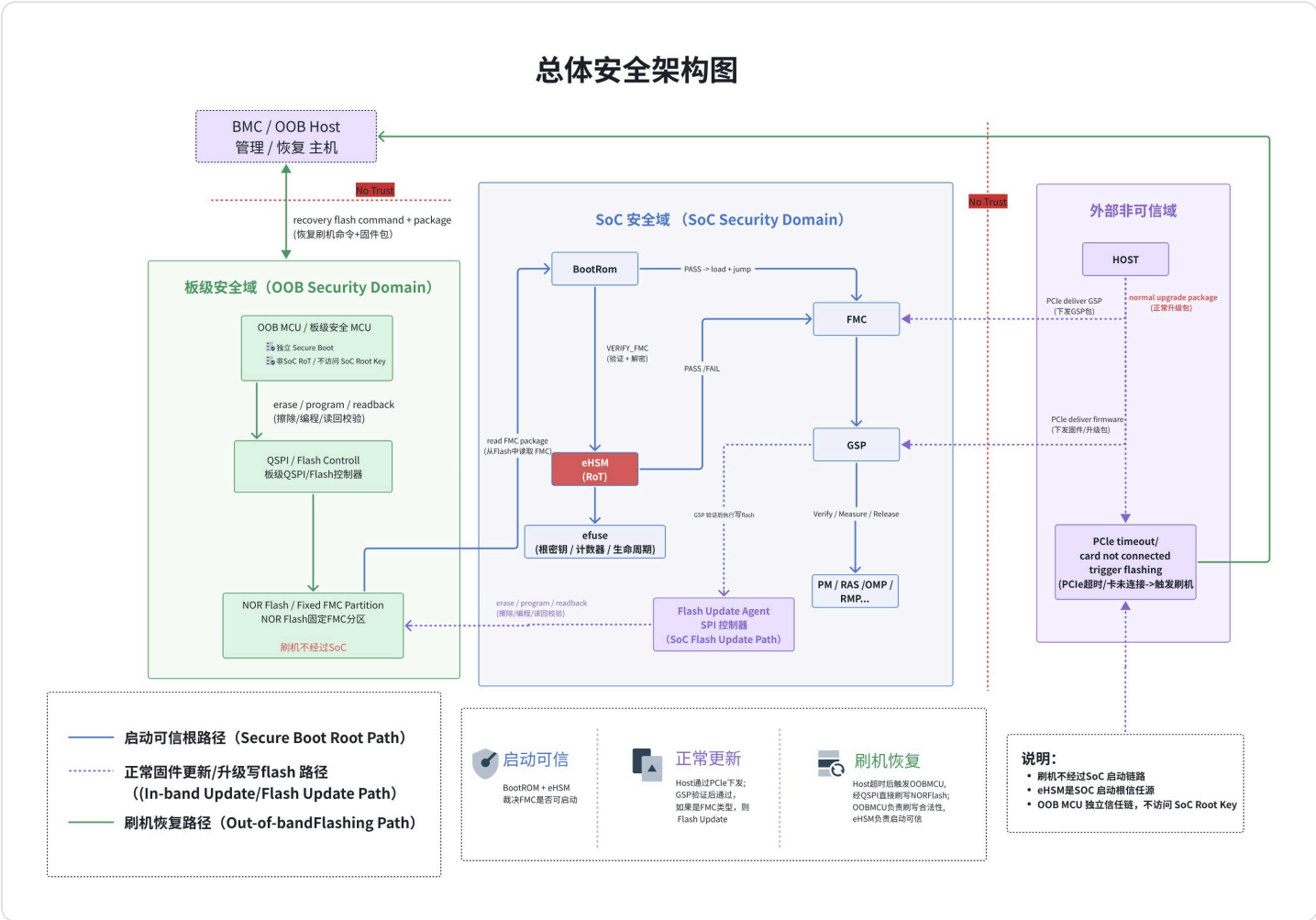


芯片安全软件方案v1.2

1. 概述

本方案面向 NGU800P 的全生命周期安全能力建设，覆盖安全启动、固件验签与解密、安全服务接口、生命周期与安全 Debug、设备远程认证、制造灌装、以及板级/OOB 安全边界。

1.1 安全总体架构



- 安全启动根路径位于 SoC 域内：BootROM -> eHSM -> FMC。
- 正常固件更新路径位于 SoC 正常运行态：Host 经 PCle 下发 GSP/Runtime 包，FMC/GSP 调 eHSM 验证。
- 验证通过后，由启动核 C908 通过 SPI/Flash 写入目标分区或 release 目标微核；Host 不拥有直接写启动关键区和 release 的权限。
- 刷机恢复路径位于板级 OOB 域：Host/BMC 触发 OOB MCU，OOB MCU 通过 SPI 直接写 NOR Flash。

- OOB MCU 具备独立信任链，并在搬运 SoC 加密镜像前校验 OOB outer signed wrapper；OOB MCU 不访问 SoC Root Key，不解密 SoC 固件。

1.2 各软件固件角色职责

角色	软件职责
BootROM (运行在启动核)	最小平台初始化、读取 secure boot/lifecycle 配置、从固定 NOR Flash 地址读取 FMC package、调用和解密、做最小 manifest 检查、记录 measurement、跳转 FMC。
FMC (运行在启动核)	是运行在启动核 C908 上的第一阶段可变固件，建立基础安全运行环境、初始化 Host 通道、接收 GSP eHSM 验证 GSP、向 GSP 转交运行期安全控制面。
GSP (运行在启动核)	后续固件验证、release 控制、measurement 维护、SPDM Responder、Debug、Provisioning 编排。
eHSM	Root Secret 使用、签名验证、解密、KDF/key unwrap、counter、lifecycle、debug auth、attestation。
Host/Driver	传输 Runtime 固件包，发起 SPDM、状态查询、升级、Debug 请求；FMC 恢复时通过 BMC/OOB 通道 recovery package。
BMC	发起带外管理、运维、升级、FMC 恢复和电源控制请求。
OOB MCU	纳入板级安全边界；承载带外代理、电源/复位、状态采集、和 SPI 刷写 NOR Flash 固定 FMC 分区。

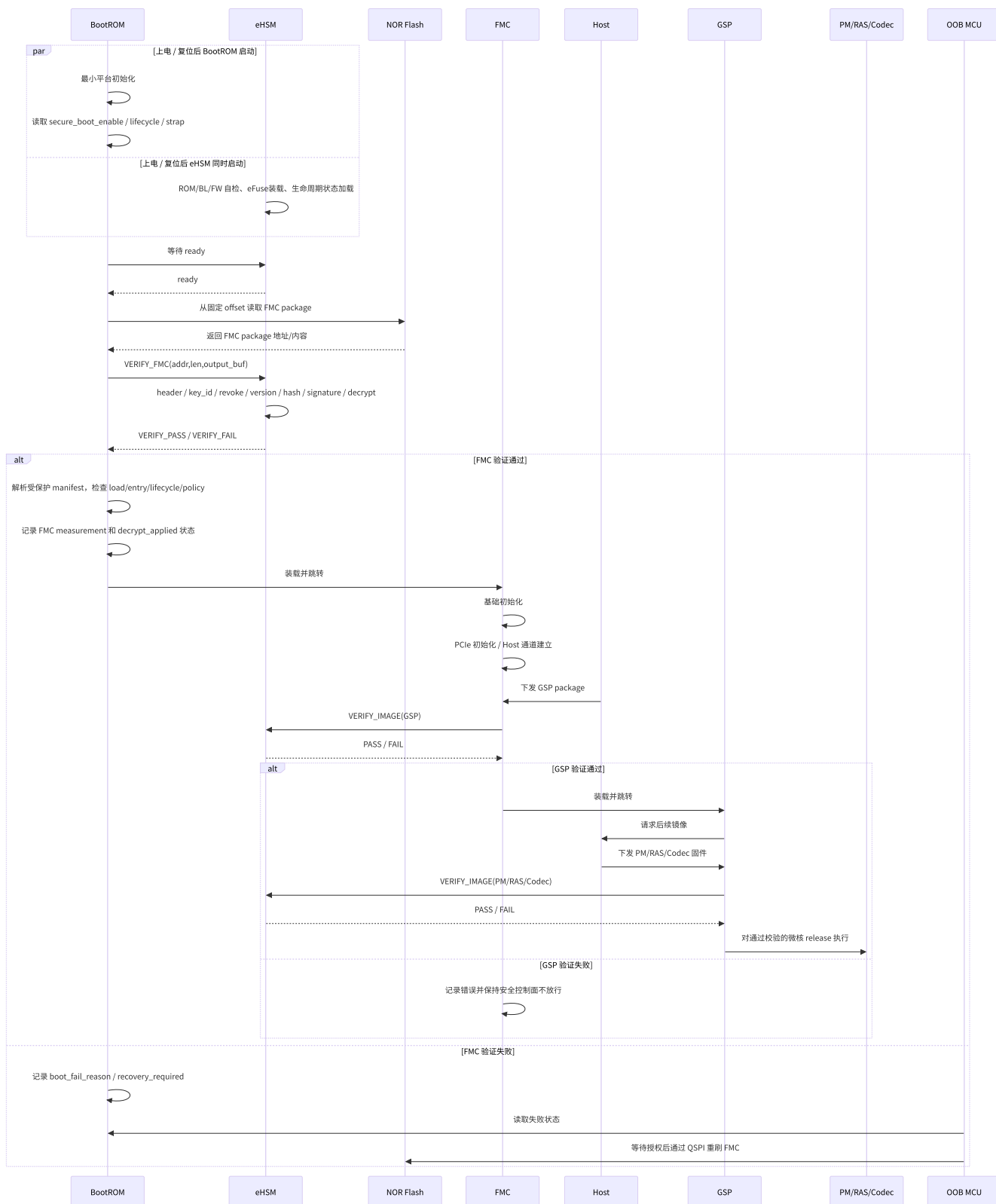
1.3 信任边界

- eHSM 内部 ROM/FW/KMU/SRAM、eHSM 专用 eFuse 属于 SoC 强安全域。
- 管理子系统启动核 C908、FMC/GSP 执行区、管理 SRAM、sec_cfg、SPIFC、Firewall 配置寄存器属于高权限受控执行域，其访问和配置需受安全启动、Firewall 和 LCS 保护。
- OOB MCU 属于板级安全边界内的受控执行体，安全级别高于 BMC/Host 链路，但不等于 SoC Root of Trust。
- Host PCIe inbound、BMC/OOB Host、SMBus/I2C/I3C、JTAG、UART、管理 DMA 都属于不可信输入面。
- release、debug enable、lifecycle、rollback counter、key revoke 等高危控制面采用“启动核 C908 执行 + eHSM 授权/裁决 + Firewall 隔离”的模型。
- 固定 FMC 分区可由 OOB MCU 通过 SPI 在授权维护流程下重写，但重写后的 FMC 必须经过 BootROM + eHSM 重新验证后才可执行。

2. 安全启动

2.1 安全启动详细时序

安全启动详细时序图



步骤说明：

1. BootROM early 阶段：

- 上电或复位后，BootROM 先做最小平台初始化，只读取启动所需的 lifecycle、strap 等配置。该阶段不信任 Host 输入，也不进入复杂更新逻辑。

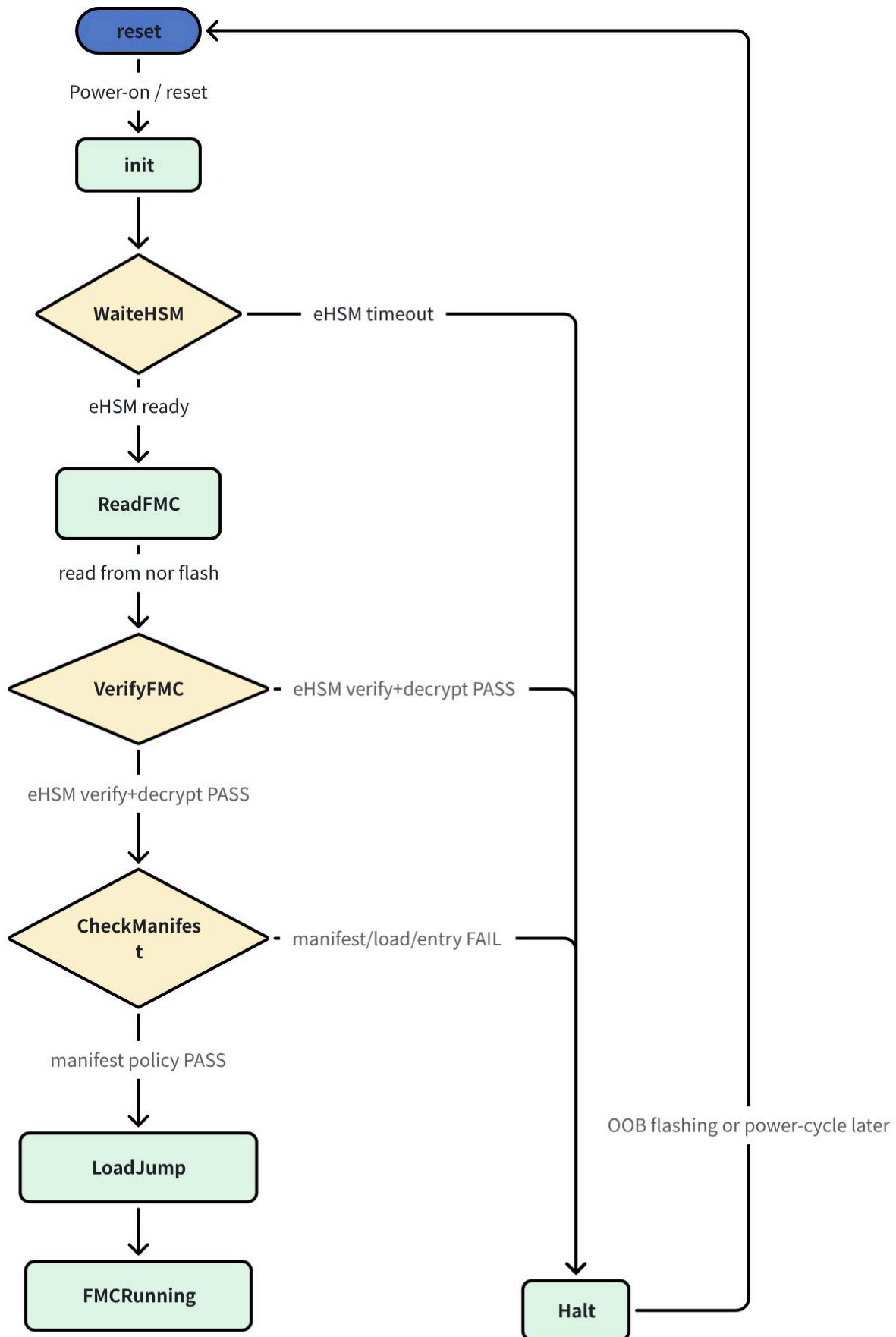
2. 拉起 eHSM 并等待 eHSM ready（寄存器状态）：

- a. BootROM 必须等待 eHSM 完成 BL/FW 自检、eFuse 装载和生命周期恢复。eHSM 未 ready 或 mailbox 超时都属于受控失败，BootROM 不允许绕过 eHSM 继续执行 FMC。
- 3. **读取固定 FMC 包：**
 - a. BootROM 只从 NOR Flash 的固定 FMC 分区读取 FMC secure package。
- 4. **FMC verify/decrypt 裁决：**
 - a. BootROM 调用 eHSM `VERIFY_IMAGE`，由 eHSM 完成 header、key、rollback、signature 和强制解密检查。正式路径中 FMC 必须签名 + 加密，验证或解密失败时 BootROM 停止跳转，可选记录诊断状态并等待复位或 OOB 刷机恢复。
- 5. **FMC manifest 与跳转：**
 - a. eHSM 返回 PASS 后，BootROM 对 protected manifest 做最小策略检查，包括 load 地址、entry 地址、lifecycle 约束和 measurement 记录。检查通过后，BootROM 将控制权交给 FMC。NGU800P 当前版本采用全局 rollback counter。eHSM 验证 FMC/GSP/Runtime/Die1 镜像时，均使用镜像 header 中的 image_counter 与 eFuse 中的 stored_global_counter 比较。
- 6. **FMC 建立 Host 通道并验证 GSP：**
 - a. FMC 完成基础初始化并建立 Host PCIe 通道。Host 只负责把 GSP 固件包传输到 staging buffer，不拥有 release 权；正式路径中 GSP 也必须签名 + 加密，FMC 调 eHSM `VERIFY_IMAGE` 完成验签、解密、rollback 和 manifest policy 检查，PASS 后才 release GSP。
- 7. **GSP 验证 Runtime 固件：**
 - a. GSP 接管运行期安全控制面后，继续接收 Host 下发的 PM/RAS/Codec/Other Runtime FW package，并调用 eHSM 完成 `VERIFY_IMAGE`、measurement 和 release policy 检查。Runtime 固件默认签名 + 加密。
- 8. **失败处理原则：**
 - a. 任一关键镜像验证失败都不允许降级为继续执行。GSP 验证失败时 FMC 向 Host 返回错误并等待重新下发；Runtime 验证失败时 GSP 拒绝 release 对应微核；FMC 验证失败时 BootROM 留在 wait reset 状态。
- 9. **OOB 刷机恢复关系：**
 - a. FMC 验证失败后的恢复不由 BootROM 自动 fallback 完成，而是等待 Host/BMC 触发 OOB MCU。
 - b. OOB MCU 通过自身独立信任链校验 OOB outer signature，校验通过后只搬运内部 eHSM native secure image package 并写入 NOR 固定 FMC 分区；重刷后的 FMC 仍需在下一次启动中由 BootROM + eHSM 完成最终 verify/decrypt 裁决。

2.2 失败处理

- FMC verify/decrypt 失败：BootROM 停止跳转 FMC，可选记录 `boot_fail_reason`、`ehsm_status`、`manifest_status` 等诊断信息，等待复位或下一次刷机后重启。
- FMC manifest policy 失败：按 FMC 验证失败处理，不跳转。
- GSP 验证失败：FMC 拒绝跳转 GSP，保持基础安全控制面，允许 Host 通过 PCIe 重新下发 GSP。
- Runtime 固件验证失败：GSP 拒绝对应微核 release，Host 可重新下发对应 Runtime 包。
- eHSM 未 ready 或 mailbox 超时：BootROM/FMC/GSP 进入受控失败路径，不允许旁路。
- Host 发现卡连接超时、PCIe 枚举失败或心跳超时：Host/BMC 触发 OOB MCU 刷机流程；该触发不依赖 BootROM 已经写入 `recovery_required`。
- USER 生命周期不允许自动降级到非安全启动。

2.3 BootROM 加载 FMC 流程



步骤说明：

1. Reset 后 BootROM 做最小初始化并等待 eHSM。
2. eHSM ready 后，BootROM 从固定 NOR Flash 偏移读取 FMC。

- 3. eHSM verify/decrypt 通过后进入 manifest 检查。
- 4. manifest、load/entry 通过后加载并跳转 FMC。
- 5. 任一失败进入 Halt，等待 OOB 刷机或电源复位后重新启动。

2.4 OOB MCU 刷写约束

项目	约束
OOB MCU 自身可信	OOB MCU 必须有独立 secure boot、升级鉴权、调试关闭、生命周期或板级授权状态。
触发条件	Host PCIe 枚举失败、连接超时、心跳超时、运维/RMA/制造授权刷机；不要求 BootROM 先 recovery_required。
SPI ownership	OOB MCU 执行刷写前必须获得 NOR Flash SPI ownership，SoC 侧不得同时访问。
写入范围	OOB MCU 只允许写固定 FMC 分区和授权允许的恢复状态区
外层校验	OOB MCU 在搬运前校验 OOB outer signed envelope，该验签流程与 OOB MCU 固件验签安全体系。
最终验证	BootROM/eHSM 必须在下一次启动时重新验证签名、解密、rollback、manifest policy 和单。
重发重刷	若掉电或写入失败，Host/BMC 可重新下发 package，OOB MCU 可重复擦写固定 FMC 分区

2.5 正常运行态更新 FMC

正常运行态更新发生在 SoC 已经完成安全启动并进入 FMC/GSP 管理态之后：

- PCIe 下发 update package 到 staging buffer
- FMC/GSP 调 eHSM VERIFY_IMAGE / VERIFY_UPDATE_AUTH
- PASS 后启动核 C908 通过 SPI/Flash 写目标分区

3. 固件包格式与验证解密流程

3.1 eHSM native package

FMC、GSP 和受保护 Runtime 固件采用 eHSM native secure image package。物理格式以 eHSM TRM 为准，NGU800P 不再定义第二套 header。



- 1. eHSM native header 承载 eHSM 原生认证/加密控制字段。
- 2. Code region 由 eHSM verify/decrypt 保护。
- 3. NGU protected manifest 位于 Code region 内，eHSM PASS 后才可被 BootROM/FMC/GSP 解析。
- 4. payload 也位于受保护 Code region 内，manifest 指示其类型、版本、load/entry 和 measurement slot。
- 5. 对 OOB recovery 场景，eHSM native package 作为内部 SoC secure image package，被 OOB outer signed envelope 包裹；OOB MCU 只校验外层 envelope，不解析内层 eHSM native header。

3.2 eHSM header 字段使用规则

字段	大小（字节）	NGU 使用规则
Signature	256	eHSM 原生签名
Public_Key	320	eHSM 原生公钥，不作为 NGU 证书区。
Encrypt_IV	16	对称加密的 IV 元数据。
Valid_Flag	4	代码是否有效标志：0x8E97645D: 代码有效； 其它值: 代码无效
Image_Type	1	镜像类型：1: SoC 2: eHSM
Plain_Flag	1	代码部分是否是明文：0: 密文； 1: 明文
Naked_Flag	1	是否需要验证签名： 0: 非裸镜像数据 1: 裸镜像数据，除了此字段、Image_Type、Valid_Flag 字段和 Code 字段，其它字段值都为全 0
Reserved	5	
Code_Size	4	代码区域的大小
Version_Counter	16	128 位的单向版本计数

3.3 NGU800P protected manifest

NGU800P 项目级字段全部放入 eHSM 认证/解密后的 Code region。BootROM/FMC/GSP 只能在 eHSM PASS 后解析 manifest。

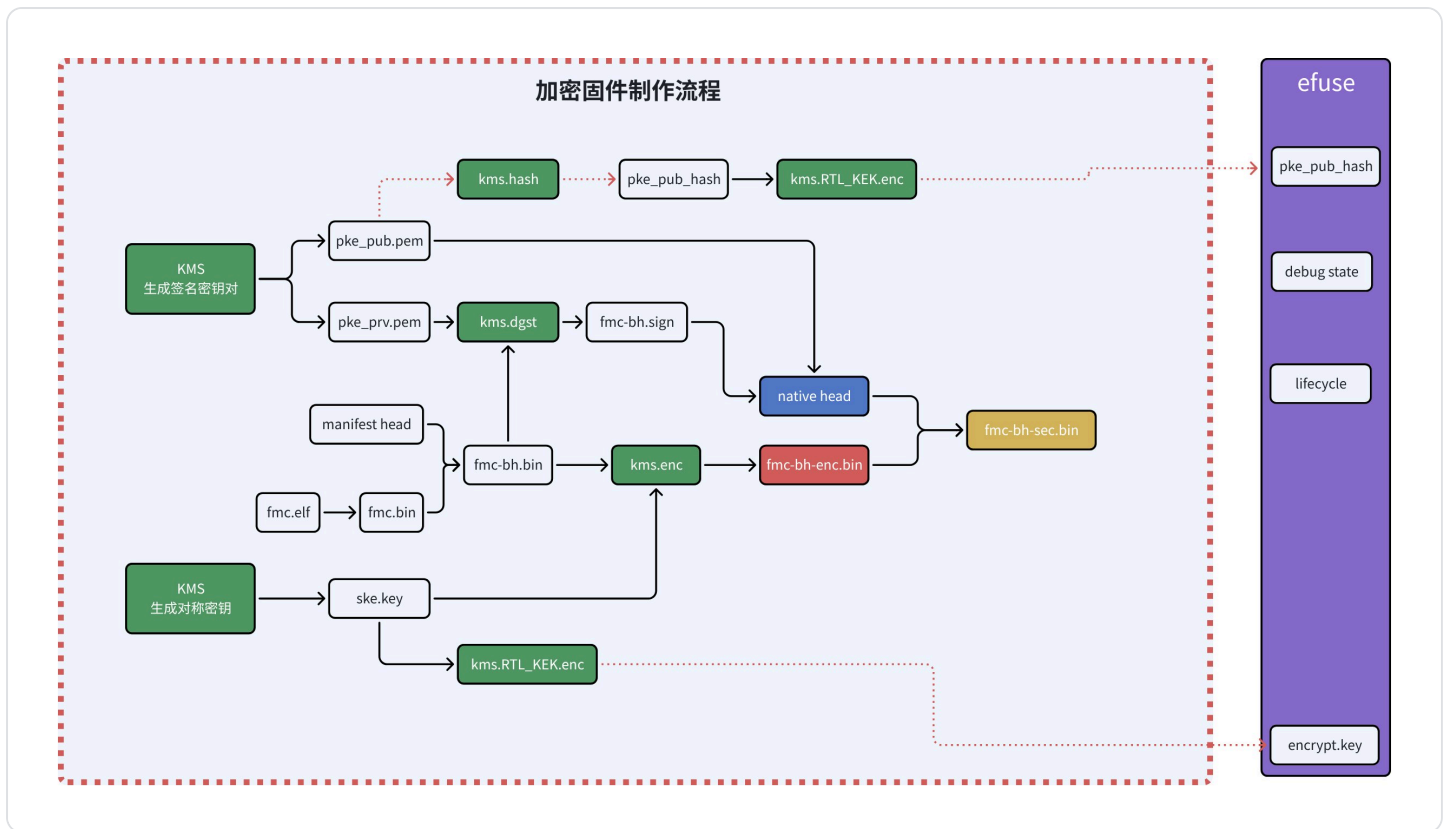
字段语义	用途
magic / manifest_version	标识 manifest ABI，拒绝未知或不兼容版本。
image_type	表达 FMC、GSP、PM、RAS、Codec 等项目级类型。
security_policy_flags	表达 sign_required、decrypt_required、rollback_required、measure_required、release_policy。
payload_offset	定位 payload，必须完全落入 Code_Size。
load_addr / entry_addr	装载和跳转地址。
version	项目级版本视图，映射到 eHSM version counter。
board_binding_policy	板级绑定结果的 attestation/release 表达。
expected_algorithm_profile	与 eHSM control field 做一致性检查，不覆盖 eHSM 算法选择。
payload_digest	payload digest 描述。

3.4 设备侧验证解密流程

1. 调用 eHSM `bl_verify_image`、`soc_verify` 或项目 wrapper。
2. eHSM 解析 native header，检查 `Image_Type`、`Plain_Flag`、`Code_Size`、`Version_Counter`、`key/signer/rollback/signature`。
3. eHSM 对 Code region 进行认证和解密，输出 plaintext Code region 到受控 output buffer。
4. 调用方在 eHSM PASS 后解析 NGU manifest。
5. 调用方检查 `ngu_image_type`、`policy flags`、`lifecycle`、`rollback`、`load/entry`、`algorithm profile`。
6. 通过后记录 measurement，装载 payload，并执行 release 或 jump。

3.5 固件包制作与设备侧校验流程图

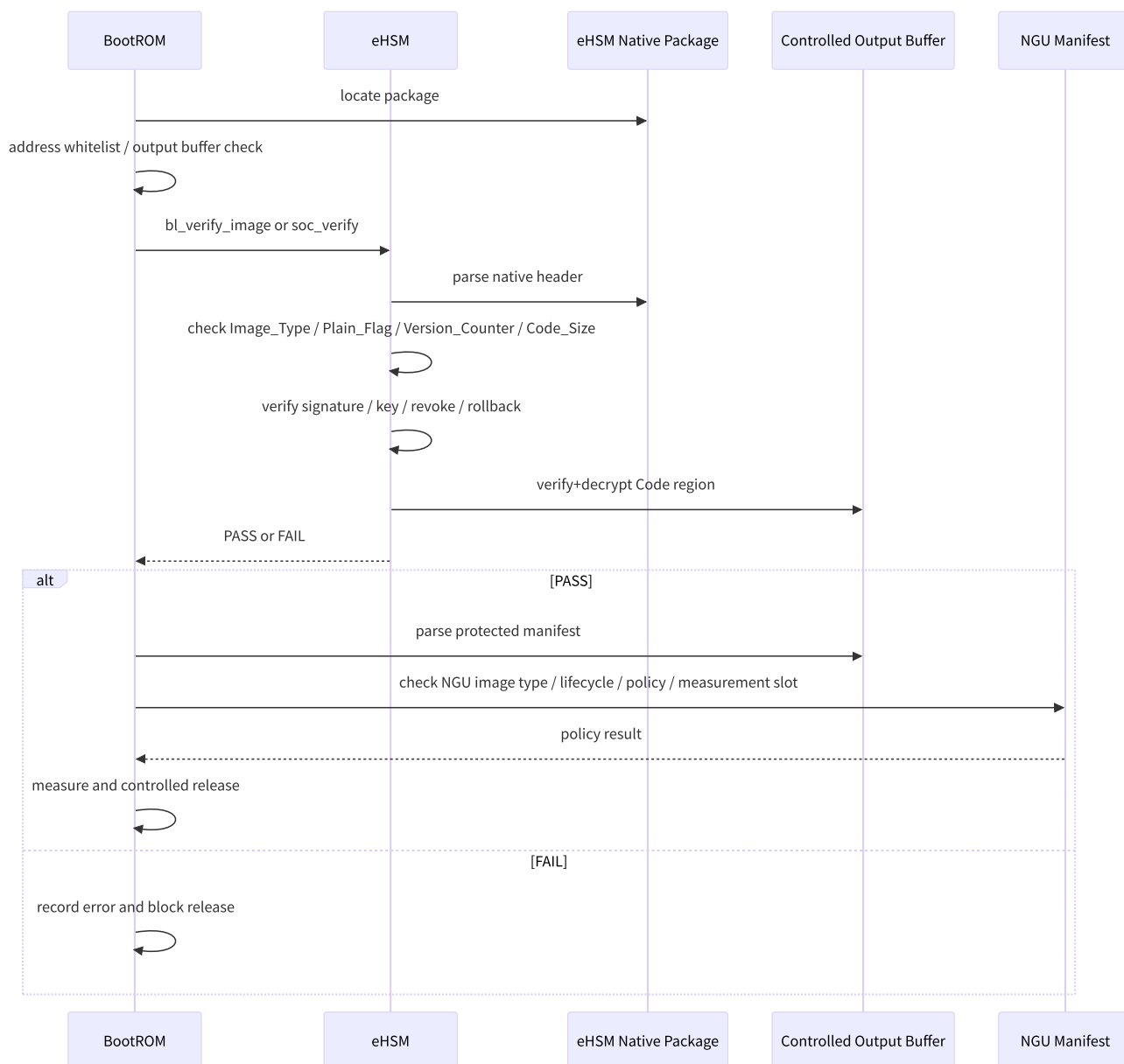
3.5.1 FMC 安全固件制作流程



1. 编译生成 fmc.bin，并准备好 manifest，生成 fmc-bh.bin；
2. 由 KMS 生成签名密钥 sign_pub_key, sign_prv_key；
3. 对 sign_pub_key 计算 pubkey_hash，并由 RTL_KEK 进行加密，在制造阶段灌装至 eFuse；
4. 计算 fmc-bh.bin 的 hash 值 (fmc-bh-hash)；
5. 使用 sign_prv_key 对 fmc-bh-hash 签名，生成 signature，填入 native header 的对应字段；
6. 由 KMS 提供对称密钥 FW_Enc_Key，对 fmc-bh.bin 加密生成 fmc-bh-enc.bin；
7. 组装 native header, 并与 fmc-bh-enc.bin 拼装成最终 fmc-bh-sec.bin(fmc 的安全固件)；
8. 对称密钥 FW_Enc_Key 经过 RTL_KEK 加密后，在制造阶段灌装至 eFuse。

3.5.2 FMC 固件验证流程

设备侧 verify/decrypt 与 manifest policy 流程



1. BootROM 首先定位 eHSM native package，并检查 image_addr、output buffer、load buffer 等地址是否落在白名单范围内。
2. BootROM 通过 bl_verify_image 或 soc_verify 调用 eHSM，并传入 package 地址、输出 buffer 地址和期望 profile。
3. eHSM 解析 native header，检查 Image_Type、Plain_Flag、Version_Counter、Code_Size 等 native 字段。
4. eHSM 完成 key、signer、rollback、signature 检查，并按 eHSM profile 对受保护 Code region 执行 verify + decrypt。
5. eHSM 将解密后的明文 Code region 输出到 Controlled Output Buffer，并向 BootROM 返回 PASS 或 FAIL。

- 6. 只有 eHSM PASS 后，BootROM 才解析 NGU Manifest；eHSM FAIL 时必须记录错误并阻断 release。
- 7. BootROM 根据 NGU Manifest 检查 image_type、lifecycle、rollback counter、measurement slot、load_addr、entry_addr 等项目级策略。
- 8. manifest policy 通过后，BootROM 记录 measurement，并执行 load + jump 或 release。
- 9. manifest policy 失败、地址越界、entry 不合法、rollback 不满足、decrypt 未执行或结果失败时，均不得继续启动或释放目标核。

4. 设备远程认证

本章定义 NGU800P 设备远程认证的软件实现方式。当前版本以 **SPDM 标准流程** 为主。

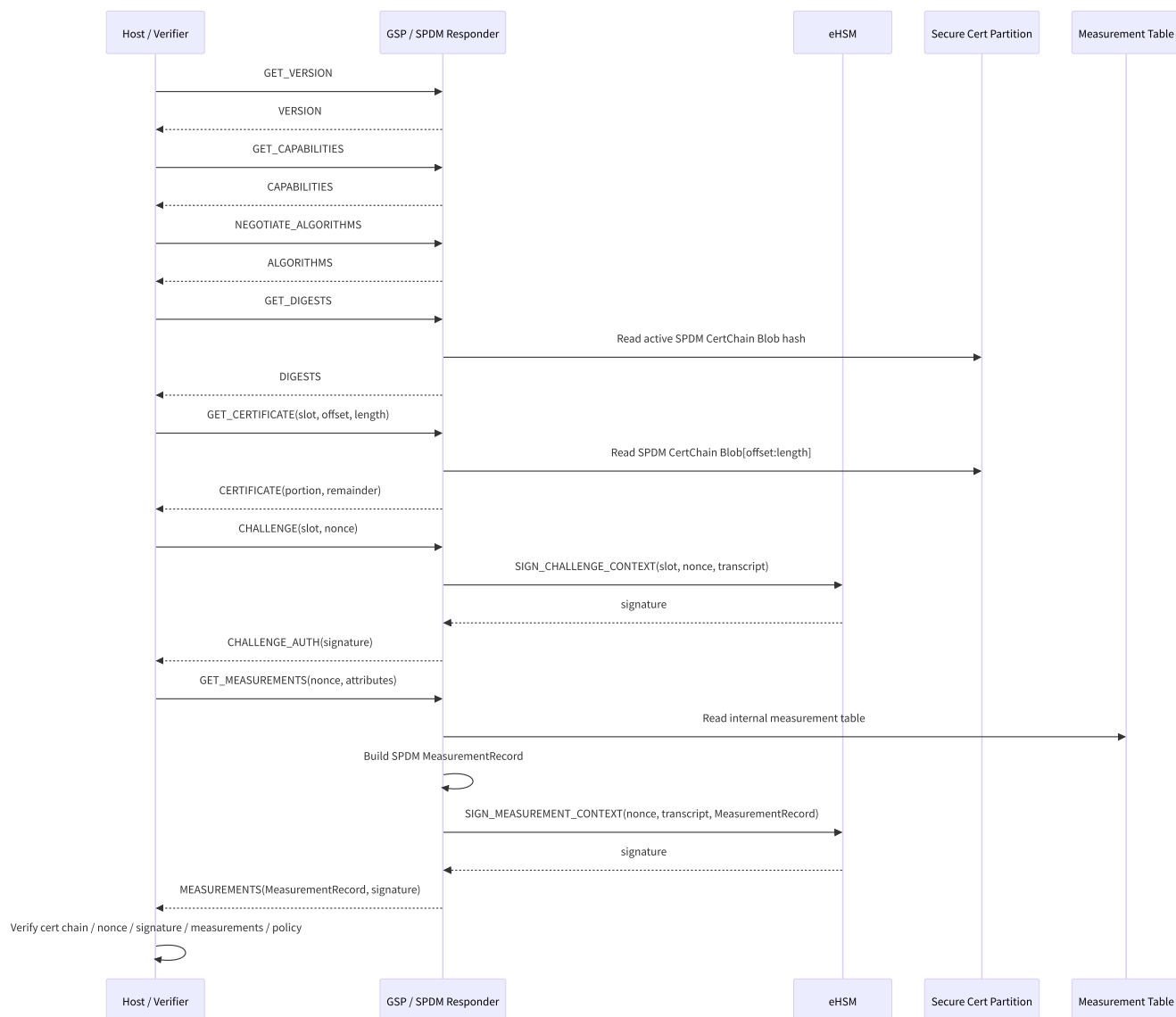
4.1 角色

角色	位置	职责
Requester	Host driver / Host security agent	发起 SPDM、获取证书、challenge、measurements、report
Responder	GSP	SPDM Responder 状态机、measurement 汇总、report 组织
Signing service	eHSM	使用 Device Attestation Private Key 对 report 或 signed register
Verifier	Host 本地策略模块或远端验证服务	校证书链、签名、nonce、measurement、lifecycle、deb

4.2 SPDM 认证流程

NGU800P 当前版本实现以下 SPDM 基础认证流程：

- 1. GET_VERSION
- 2. GET_CAPABILITIES
- 3. NEGOTIATE_ALGORITHMS
- 4. GET_DIGESTS
- 5. GET_CERTIFICATE
- 6. CHALLENGE
- 7. GET_MEASUREMENTS



1. `GET_VERSION`、`GET_CAPABILITIES`、`NEGOTIATE_ALGORITHMS` 用于协商 SPDM 版本、能力和算法。
2. `GET_DIGESTS` 返回 certificate slot 的证书链摘要。
3. `GET_CERTIFICATE` 按 offset、length 分片返回对应 slot 中的 SPDM CertChain Blob。
4. `CHALLENGE` 用于设备身份挑战认证，eHSM 使用 Device Attestation Private Key 对 challenge 上下文签名。
5. `GET_MEASUREMENTS` 用于获取设备 measurement record；需要签名时，GSP 请求 eHSM 绑定 nonce 和 transcript 后签名。
6. 当前版本不进入 `KEY_EXCHANGE` / `FINISH`，不建立 SPDM secure session。
7. Verifier 不应只校证书或签名，还必须校验 measurement、lifecycle、debug、counter 和固件保护策略。

4.3 设备内部 measurement table

设备内部可以维护私有 measurement table，用于保存 BootROM、FMC、GSP、Runtime和 SoC 安全状态。该结构仅供 GSP 内部使用，不直接作为 SPDM 响应返回给 Host。

代码块

```
1  typedef struct {
2      uint32_t magic;                /* NGU_MEAS_TABLE_MAGIC */
3      uint16_t version;              /* measurement table 结构版本 */
4      uint16_t header_len;           /* 本 header 长度 */
5      uint32_t total_len;            /* 整个 measurement table 长度 */
6
7      uint8_t device_uuid[16];       /* 设备 UUID */
8      uint8_t chip_id[16];           /* 芯片唯一 ID */
9
10     uint32_t lifecycle_state;        /* 当前生命周期 */
11     uint32_t debug_enable_state;     /* Debug 大开关状态，不区分 scope */
12     uint32_t stored_global_counter; /* eFuse 中全局 rollback counter */
13
14     uint32_t cert_slot_id;           /* 当前 SPDM certificate slot */
15     uint32_t cert_chain_digest_algo;
16     uint32_t cert_chain_digest_len;
17     uint8_t cert_chain_digest[NGU_HASH_MAX_LEN];
18
19     uint32_t fw_entry_count;         /* 固件 measurement entry 数量 */
20     uint32_t state_entry_count;     /* 状态 measurement entry 数量 */
21
22     uint32_t reserved[8];
23 } measurement_table_header_t;
```

固件 measurement entry:

代码块

```
1  typedef enum {
2      NGU_FW_TYPE_BOOTROM = 0x0001,
3      NGU_FW_TYPE_FMC     = 0x0002,
4      NGU_FW_TYPE_GSP     = 0x0003,
5      NGU_FW_TYPE_PM      = 0x0004,
6      NGU_FW_TYPE_RAS     = 0x0005,
7      NGU_FW_TYPE_CODEC   = 0x0006,
8      NGU_FW_TYPE_DIE1_FW = 0x0100,
9  } ngu_fw_type_t;
10
11  typedef struct {
12      uint32_t fw_type;                /* ngu_fw_type_t */
```

```

13     uint32_t die_id;                /* 0: Die0, 1: Die1 */
14     uint32_t flags;                 /* boot/runtime/recovery 等 */
15
16     uint32_t hash_algo;
17     uint32_t hash_len;
18     uint8_t hash[NGU_HASH_MAX_LEN]; /* 对加密固件, 推荐为 plaintext hash */
19
20     uint32_t image_counter;         /* 镜像 header 中的 counter */
21
22     uint32_t key_id;                /* 本次验证使用的 key slot */
23     uint32_t signer_id;             /* 本次验证使用的 signer / public key hash
slot */
24
25     uint32_t verify_result;         /* PASS / FAIL / NOT_VERIFIED */
26     uint32_t release_state;         /* NOT_RELEASED / RELEASED / FAILED */
27
28     uint32_t load_addr;             /* 可选 */
29     uint32_t entry_addr;            /* 可选 */
30     uint32_t reserved[4];
31 } ngu_meas_fw_entry_t;

```

SoC 状态 measurement entry:

代码块

```

1  typedef struct {
2      uint32_t lifecycle_state;
3      uint32_t debug_enable_state; /* Debug 大开关状态, 不区分 scope */uint32_t
anti_rollback_enable;
4
5      uint32_t stored_global_counter; /* eFuse 中全局 rollback counter */uint32_t
firewall_state;
6      uint32_t oob_recovery_state;
7      uint32_t boot_fail_reason;
8
9      uint32_t ehsm_status;
10     uint32_t ehsm_error;
11
12     uint32_t die_num;
13
14     uint32_t reserved[8];
15 } meas_soc_state_entry_t;

```

4.4 SPDM MEASUREMENTS response 结构视图

SPDM 对外返回的是 `MEASUREMENTS response`，不是内部 `measurement table`。

`MEASUREMENTS response` 的逻辑组成如下：

代码块

```
1  typedef struct {
2      spdm_message_header_t header;
3
4      /*
5       * SPDM MEASUREMENTS response 的元数据。
6       * 不同 SPDM 版本中部分字段可能通过 header.param1/param2 表达；
7       * 此处是文档视图，用于说明整体组成。
8       */uint8_t number_of_blocks;
9
10     /*
11      * SPDM 中 MeasurementRecordLength 通常是 3 bytes 长度字段。
12      * 实现时必须按选定 SPDM 版本编码。
13      */uint8_t measurement_record_length[3];
14
15     /*
16      * variable length:
17      * measurement_record = 多个 spdm_measurement_block_view_t 顺序拼接。
18      */uint8_t measurement_record[];
19
20     /*
21      * optional:
22      * opaque data / signature follow according to SPDM version/profile.
23      */
24 } spdm_measurements_response_view_t;
```

`MeasurementRecord` 由多个 `MeasurementBlock` 顺序拼接组成。外层 block 格式遵循 SPDM，内部 `measurement value` 可使用 NGU800P 自定义 payload。

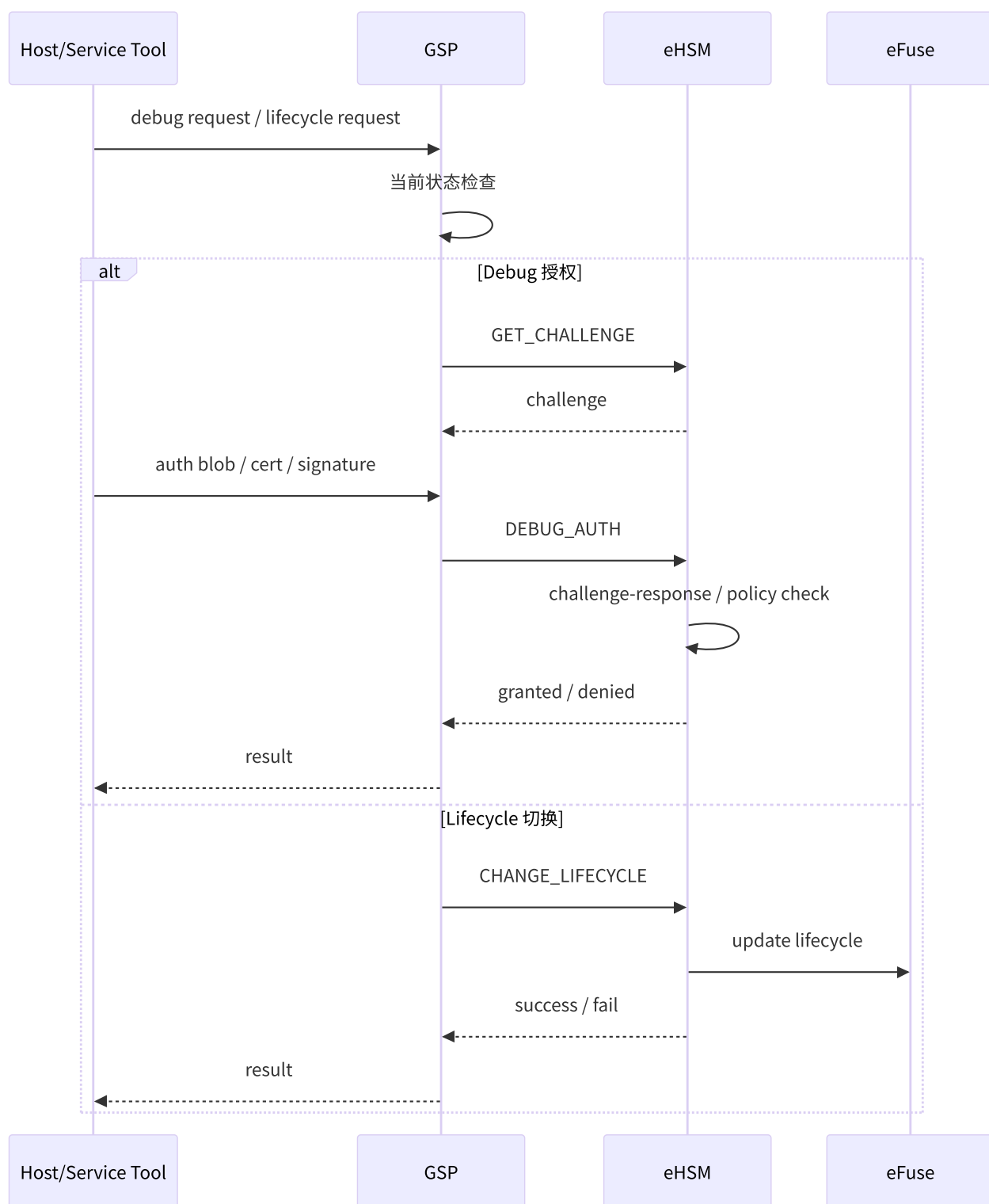
5. 生命周期与安全 Debug

5.1 生命周期模型

状态	语义	启动策略	Debug 策略
TEST	裸片/封测/初测	可开放测试路径，不等价量产	开发授权
DEV	开发/EVB	可安全/非安全启动	开发授权
MANUFACTURE	正式制造	优先安全启动	有限受控
USER	量产交付	强制安全启动	默认关闭
DEBUG	受权返修	受控签名启动	授权后有限开放
DESTROY	DEST	销毁模式，不提供安全服务	关闭

5.2 Debug 授权流程

Debug 授权与生命周期切换时序



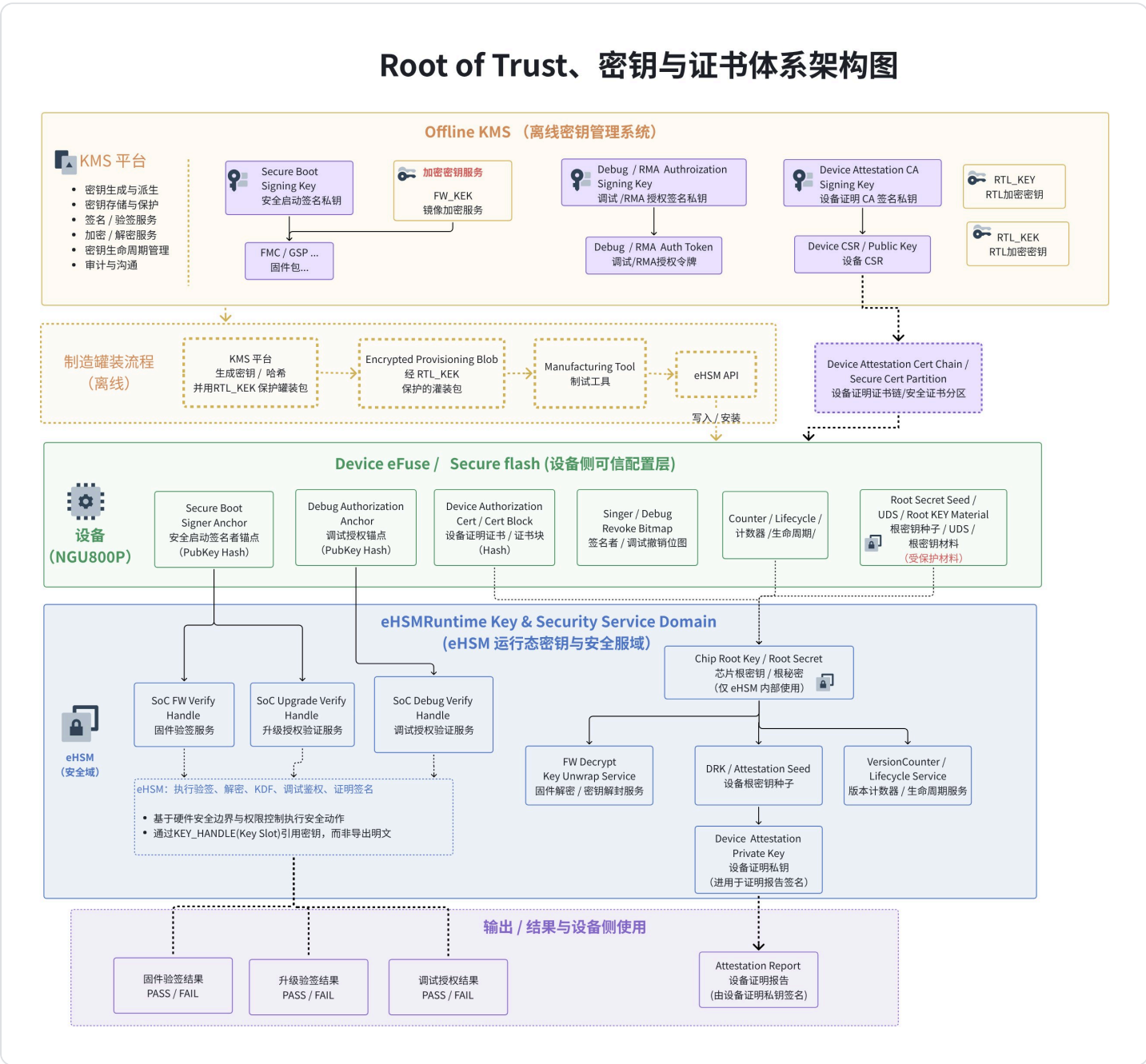
1. Host/BMC 向 GSP 发起 debug request。
2. GSP 检查 lifecycle、当前 secure boot 状态和请求格式。
3. GSP 调用 eHSM `GET_CHALLENGE`。
4. 工具提交 Debug token，对 challenge、UID 签名。
5. GSP 调用 eHSM `DEBUG_AUTH`。

- 6. eHSM 校验 challenge、证书、lifecycle 和过期策略。
- 7. 通过后返回 granted scope、expire policy、audit id。
- 8. GSP 根据鉴权结果打开或关闭统一 debug enable 大开关，并设置自动关闭条件。
- 9. 超时、复位、生命周期切换、安全错误或显式 `CLOSE_DEBUG` 时关闭并记录审计。

6. 密钥、证书与制造灌装

本章定义 NGU800P 安全方案中的 Root of Trust、密钥体系、证书体系、eFuse 持久化对象、eHSM 运行态密钥使用方式，以及制造阶段的灌装流程

6.1 密钥证书体系



1. Offline KMS 是外部信任源，负责保存签名私钥、授权私钥、设备证明 CA 私钥，并提供固件加密、CEK 服务和制造阶段 provisioning blob 生成能力。
2. 外部签名私钥不进入设备。设备侧保存的是 signer anchor、authorization anchor、debug anchor、lifecycle、counter、root material 和证书信息。
3. 需要写入 eFuse key slot 的 key/hash，由 KMS 生成后使用 RTL_KEK 加密成 provisioning blob，再由制造工具调用 eHSM `install_encrypt_key` 写入设备。
4. eFuse 是设备侧可信配置层，负责持久保存“设备信谁、允许什么策略、当前处于什么安全状态”。
5. eHSM 是运行态安全服务执行域。eHSM 上电后加载、解密或映射 eFuse 中的对象，形成内部 `KEY_HANDLE`、key slot 和安全上下文。
6. eHSM 对外提供固件验签、固件解密、升级授权验证、Debug/RMA 鉴权、计数器/生命周期服务和证明报告签名服务。
7. Device Attestation Private Key 只用于签署证明报告，不用于固件验签、固件解密、升级授权或 Debug 授权。
8. 完整设备证书链存放在 flash 的安全证书分区。
9. OOB MCU 不访问 SoC Root Key、不访问 eHSM 内部密钥，也不替代 eHSM 判断刷入的 FMC 是否可信。

6.2 OTP Key 安装 API 策略

制造阶段写入 eFuse 的密钥类对象，主要使用两类 eHSM OTP Key 安装 API：

代码块

```
1  install_random_key
2  install_encrypt_key
```

这两类接口只用于持久 OTP Key Slot 类对象，不用于完整证书链、生命周期、控制字段或普通运行态 session key。

6.2.1 `install_random_key`

`install_random_key` 用于由 eHSM 内部生成随机密钥，并使用对应 Root Key 加密后写入 eFuse。

- 适用对象：

对象	说明
设备内部随机对称密钥	不希望密钥在 KMS/HSM 外部明文出现
Device Attestation Private Key	推荐设备内生成私钥，私钥不出设备
device-local secret	设备本地保护或派生使用

• 不适用对象：

对象	原因
Secure Boot Signer Anchor	需要安装外部签名公钥 hash，不能随机生成
Debug Authorization Anchor	需要安装外部 Debug/RMA 授权公钥 hash
cert hash / issuer anchor	是外部证书系统确定的摘要或锚点

6.2.2 install_encrypt_key

install_encrypt_key 用于安装外部 KMS/HSM 指定的密钥、私钥或公钥 hash。

制造阶段流程如下：

- 1. KMS/HSM 生成或派生 key/hash
- 2. 计算 CRC
- 3. 组装 key/hash + CRC + padding
- 4. 使用 RTL_KEK 加密
- 5. 形成 encrypted provisioning blob
- 6. 制造工具调用 eHSM install_encrypt_key
- 7. eHSM 内部使用 RTL_KEK 解密
- 8. eHSM 校验 key_type / key_slot_id / CRC / lifecycle 权限
- 9. eHSM 使用 Chip Root Key 或 Device Root Key 重新加密
- 10. 写入 eFuse Key Slot

• 适用对象：

对象	推荐原因
SOC Encrypt Key	固件 Code 区解密密钥，由 KMS/HSM 生成或派生
SOC FW Verify Key / PubKey Hash	安全启动 signer 公钥 hash
SOC Debug Verify Key / PubKey Hash	Debug/RMA 授权 signer 公钥 hash
User Auth Key	USER 模式受控操作鉴权 key

6.3 设备证明密钥与证书策略

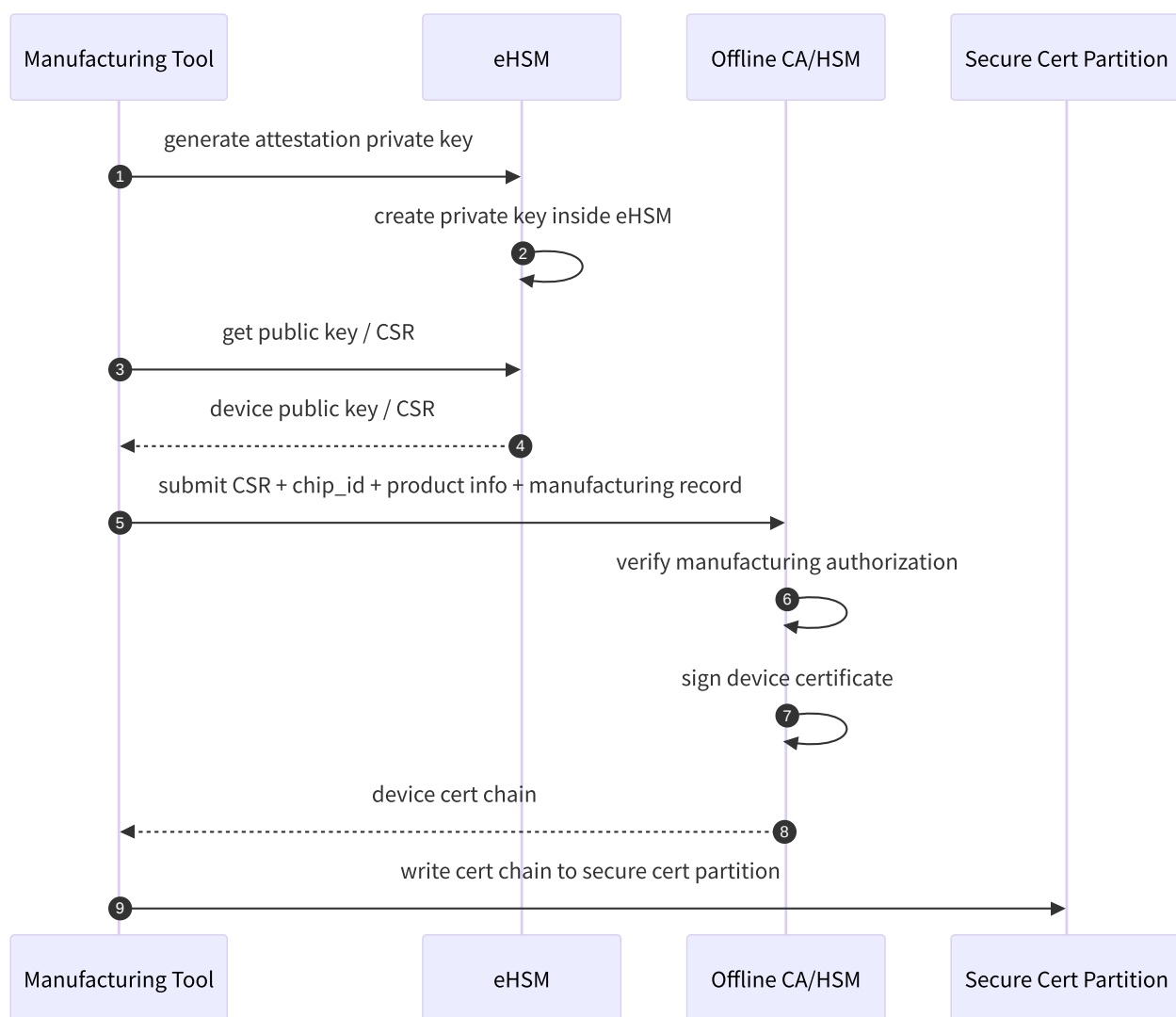
6.3.1 设备证明对象关系

对象	位置	说明
Device Attestation Private Key	eHSM 内部 key slot / secure storage	用于签署 attestation report
Device Attestation Public Key	CSR / 证书中	可公开
Device Attestation Cert	Secure cert partition	证明设备公钥属于合法设备
Device Attestation CA Signing Key	Offline CA/HSM	签发设备证书，不进入设备
Root CA Cert	Host/Verifier 预置	验证端信任锚

6.3.2 证书生成策略

设备证明私钥由 eHSM 内部生成，私钥不出设备。制造阶段仅导出公钥 或 CSR，由离线 HSM 签发设备证明证书。

Device Attestation Key / CSR / 证书签发流程图



步骤说明：

1. 制造工具请求 eHSM 内部生成设备证明私钥。
2. 私钥在 eHSM 内部生成和保存，不出设备。
3. eHSM 输出设备公钥或 CSR。
4. 制造工具将 CSR、chip_id、product info 和制造记录提交给离线 HSM。
5. HSM 校验制造授权后签发设备证书。
6. 制造工具将设备证书链写入安全证书分区。
7. 运行时 eHSM 使用设备证明私钥签署 attestation report。

6.4 证书链与证书存储策略

6.4.1 总体策略

NGU800P 当前版本采用静态 X.509 设备证明证书链方案，不采用 DICE 动态证书链。**用于 SPDM 设备认证的证书链保存在 flash 中，eHSM 内部保存 Device Attestation Private Key**，运行时由 eHSM 使用该私钥对 attestation report 或 SPDM challenge 上下文进行签名。

证书链存储和对外返回尽量直接对齐 SPDM certificate chain 格式，避免在 SPDM responder 中引入额外的私有转换逻辑。

NGU800P 当前版本设备侧证书链采用两级证书：**Intermediate CA Cert 和 Device Attestation Cert。**

其中：

- Root CA Cert：
 - 根 CA 证书，对应整条证明证书链的最高信任根。
 - Verifier 最终是否信任该 Root CA，以 Verifier 本地 trust store 为准。
- Intermediate CA Cert：
 - 中间 CA 证书，用于签发 Device Attestation Cert。
 - 可按产品线、客户、批次或设备类型划分。
- Device Attestation Cert：
 - 设备证明证书，包含 Device Attestation Public Key、device_uuid、
 - chip_id、product_id 等设备身份信息。

6.4.2 SPDM 证书链格式

设备侧 Secure Cert Partition 中每个有效证书槽保存一份 SPDM CertChain Blob。该 Blob 直接作为 SPDM GET_CERTIFICATE 响应中的数据源。

当前版本按 SPDM certificate chain 格式组织：

字段	长度	说明
Length	2 bytes	整个 certificate chain 的总长度，包含 Length、Reserved、RootHash 和 Certificates 字段，采用 little endian。
Reserved	2 bytes	保留字段，写 0。
RootHash	H bytes	Root CA Cert DER 的 Hash。H 为 SPDM 算法协商阶段选定 Hash 算法的输出长度，例如 SHA-256 为 32 bytes，SHA-384 为 48 bytes。
Certificates	variable	一个或多个 ASN.1 DER 编码的 X.509 v3 证书按顺序拼接。DER 编码本身可解析出单张证书长度，因此不需要额外 cert_count 或 cert_len 字段。

对于 NGU800P 来说也就是：

代码块


```

1  SPDM CertChain Blob =
2      Length
3      Reserved
4      RootHash = Hash(Root_CA_Cert.der)
5      Intermediate_CA_Cert.der
6      Device_Attestation_Cert.der

```

证书顺序必须满足：

1. 第一个证书应由 Root CA Cert 签发，或者第一个证书本身就是 Root CA Cert。
2. 后续每个证书应由前一个证书签发。
3. 最后一张证书必须是 leaf certificate。
4. 对于 NGU800P，最后一张证书为 Device Attestation Cert，其中包含 Device Attestation Public Key。

由于 NGU800P 设备侧不保存 Root CA Cert，因此 `Certificates` 字段从 `Intermediate CA Cert` 开始，`RootHash` 用于绑定 Verifier 本地 trust store 中的 Root CA Cert。

6.4.3 设备侧证书槽存储

Secure Cert Partition 支持多个（至少两个）SPDM certificate slot。每个 slot 对应一条 SPDM certificate chain。每个 slot 带有一个 cert_slot_metadata

代码块

```

1  typedef struct {
2      uint32_t magic;                /* 固定为 NGU_CERT_SLOT_MAGIC */
3      uint16_t version;              /* metadata 结构版本 */
4      uint16_t header_len;           /* 本 header 长度 */
5
6      uint32_t slot_id;               /* 证书槽编号，对应 SPDM SlotID */
7      uint32_t state;                /* EMPTY / VALID / ACTIVE / STAGING */
8      uint32_t flags;                /* 预留标志位，例如 recovery、factory、
customer_owned */
9
10     uint32_t cert_format;            /* 固定为 NGU_CERT_FORMAT_SPDM_CHAIN */
11     uint32_t spdm_version;           /* 例如 0x00010200 表示 SPDM 1.2 */
12
13     uint32_t cert_chain_offset;      /* SPDM CertChain Blob 在当前 slot 内的偏移 */
14     uint32_t cert_chain_len;         /* SPDM CertChain Blob 长度，应与 Blob 内
Length 字段一致 */
15
16     uint32_t cert_chain_hash_algo;   /* cert_chain_hash 使用的 hash 算法 */
17     uint32_t cert_chain_hash_len;   /* cert_chain_hash 有效长度 */

```

```

18     uint8_t  cert_chain_hash[64];    /* Hash(SPDM CertChain Blob), 用于写入后校
    验 */
19
20     uint32_t rollback_counter;    /* 可选, 证书链更新反回滚计数 */
21     uint8_t  reserved[16];        /* 预留扩展, 保持结构对齐 */
22 } ngu_cert_slot_metadata_t;

```

slot metadata 仅用于设备内部 Flash 分区管理、证书链更新和完整性检查，不应作为 SPDM GET_CERTIFICATE 的一部分返回给 Host。

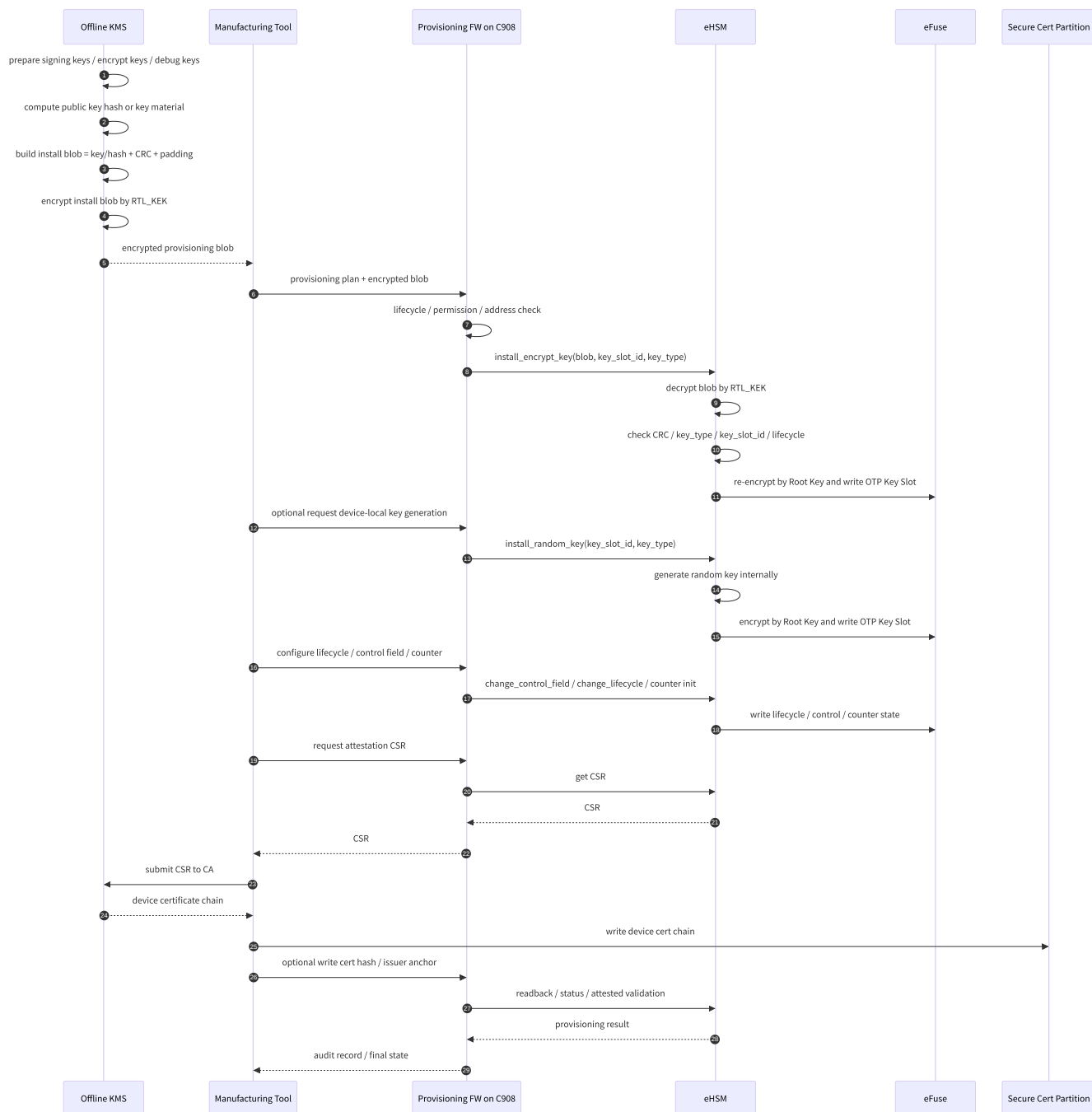
6.4.4 证书链生成与烧录

制造阶段证书链生成流程如下：

1. eHSM 内部生成 Device Attestation Private Key。
2. eHSM 导出 Device Attestation Public Key。
3. KMS 使用 Intermediate CA Private Key 签发 Device Attestation Cert。
4. KMS 系统准备 Intermediate CA Cert 和 Device Attestation Cert。
5. 计算 RootHash = Hash(Root_CA_Cert.der)。
6. 按 SPDM CertChain Blob 格式组装证书链（`device_cert_chain_spdm.bin`）：
 - Length
 - Reserved
 - RootHash
 - Intermediate_CA_Cert.der
 - Device_Attestation_Cert.der
7. 制造工具将 SPDM CertChain Blob 写入 Secure Cert Partition 的 active cert slot。
8. 写入后执行 readback，并校验 CertChainHash。

6.5 制造灌装总流程

制造灌装总流程图



图下步骤说明:

1. KMS 准备启动、升级、Debug、加密相关密钥。
2. 对于非对称验签 key，KMS 计算 signer public key hash。
3. 对于对称加密 key，KMS 生成或派生 `SOC Encrypt Key`、`SOC Upgrade Encrypt Key` 等。
4. KMS 构造 key/hash + CRC + padding 的安装 blob。
5. KMS 使用 RTL_KEK 对安装 blob 加密。
6. 制造工具将 encrypted provisioning blob 下发给 运行在启动核 C908 上的 Provisioning FW。

7. SEC 检查 lifecycle、权限和参数边界后调用 eHSM `install_encrypt_key`。
8. eHSM 使用 RTL_KEK 解密安装 blob，并校验 CRC、key type、slot id 和 lifecycle 权限。
9. eHSM 使用 Chip Root Key 或 Device Root Key 重新加密，并写入 eFuse key slot。
10. 对于设备内生成的私钥或随机 key，制造工具调用 `install_random_key`。
11. 生命周期、控制字段和版本计数器不走密钥安装 API，而通过独立控制接口配置。
12. 设备证明证书链写入安全证书分区。
13. 灌装完成后执行 readback/status/attested validation，并生成制造审计记录。

6.6 安全约束

1. RTL_KEK 只用于制造阶段密钥安装 blob 的传输保护，不作为运行态业务密钥使用。
2. `install_encrypt_key` 的输入 blob 不得在工站本地明文落盘。
3. 进入 USER 生命周期前，应关闭 TEST/DEV/MANU 模式下的任意 efuse 写能力。
4. USER 模式下不得允许重新安装启动验签 key、固件解密 key、升级授权 key。
5. Debug/RMA 授权 key 应与启动验签 key、升级授权 key 分离。
6. Device Attestation Private Key 不得导出，不得用于固件验签、固件解密、升级授权或 Debug/RMA 授权。
7. OOB MCU 不得访问 SoC eHSM Root Key、SOC Encrypt Key、SOC FW Verify Key、Device Attestation Private Key 等密钥材料。
8. 正常更新写 Flash 前必须完成 eHSM 验证。
9. OOB MCU 刷机完成后的镜像，仍必须在下一次 SoC 启动时由 BootROM + eHSM 验证通过后才能执行。
10. 所有 key install、lifecycle change、debug auth、firmware update 和 OOB recovery 事件都应进入审计记录。
11. USER 前必须清理测试 key、测试证书、测试 signer anchor 和开发调试白名单。

7. 密钥轮换管理与鉴权策略

当前支持以下三类 SoC Key 轮换：

- SoC FW Verify Key
- SoC FW Encrypt Key
- SoC Debug Key

密钥轮换采用 **新 Key Slot + Active Bitmap** 的方式实现。eFuse 中新增 Active Bitmap，用于标识当前生效的

7.1 Active Bitmap

eFuse 中新增 `soc_key_bitmap`，用于标识当前生效的 Key slot。

eHSM ROM 上电后执行：

- 读取 soc_key_bitmap
 - 校验 Bitmap 是否合法
 - 选择 FW Verify / FW Encrypt / Debug Key Slot
 - 装载到 KMU 或建立运行态 Key Handle
- Active Bitmap 应满足以下要求：
 - 支持 eFuse 单向烧写；
 - 能唯一确定当前 Active Key slot；
 - 上电后由 eHSM ROM 统一解析；
 - Bitmap 更新后不得自动回退到旧 Key Set

7.2 OTP 烧写通路 with 生命周期约束

eFuse 烧写权限受芯片生命周期控制，支持两条烧写通路：

1. JTAG 硬件烧写通路；
2. eHSM Core 软件烧写通路。

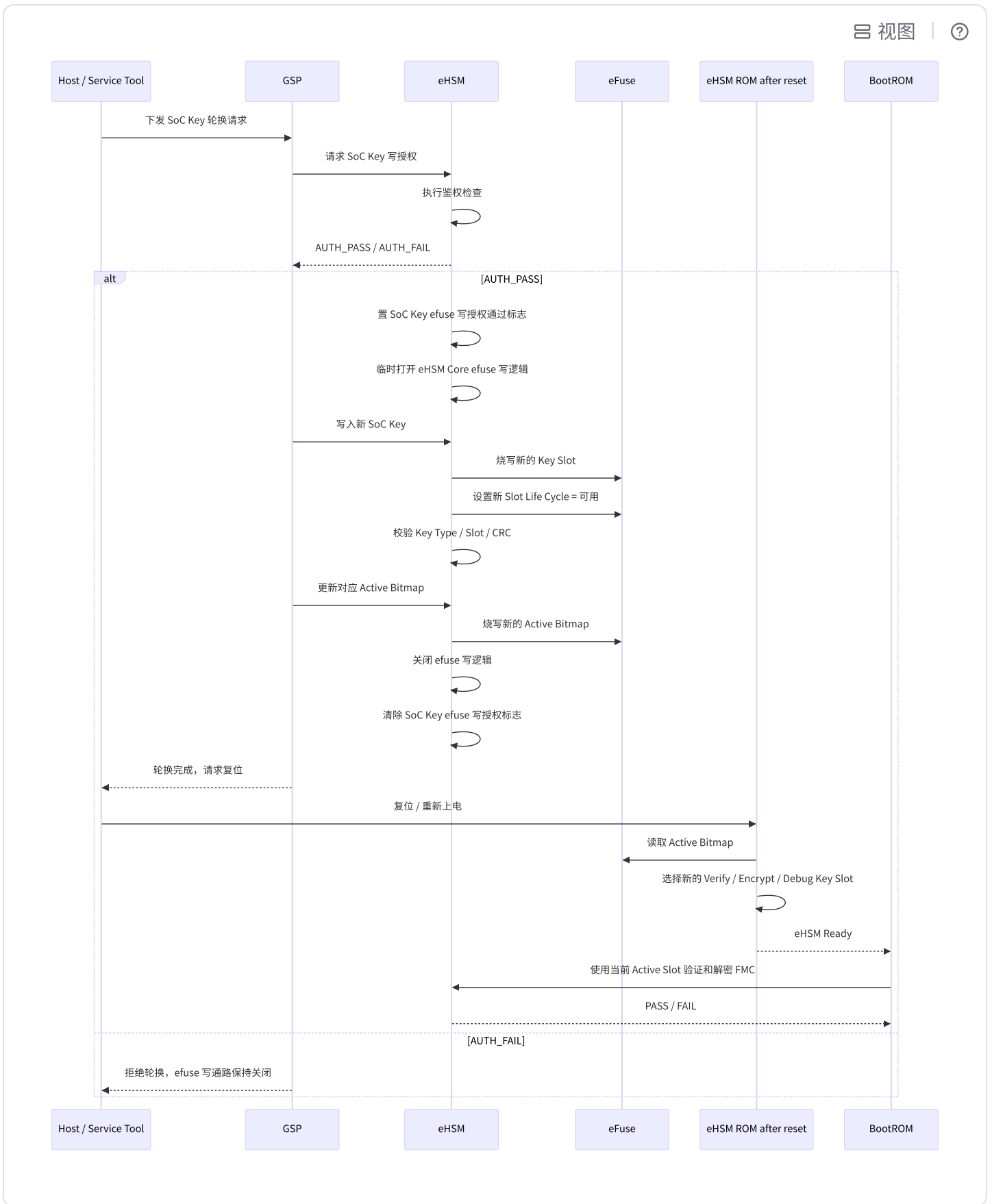
生命周期	JTAG 写 efuse	eHSM Core 写 efuse
TEST / DEV / MANU	按测试或制造策略开放	按测试或制造策略开放
USER	默认关闭，授权后临时开放	默认关闭，授权后临时开放
DESTROY	永久关闭	永久关闭

USER 生命周期下：

- JTAG efuse 写通路默认由硬件关闭，鉴权通过后临时打开；
- eHSM Core efuse 写逻辑默认关闭，eHSM 鉴权通过后通过内部授权状态临时打开；
- 操作完成、失败、超时或复位后，efuse 写权限必须重新关闭；
- 正常 SoC Key 轮换建议通过 Mailbox + eHSM Core 软件通路完成；
- JTAG 通路主要用于 TEST、MANU 或受控 DEBUG。

7.3 USER 生命周期下的密钥轮换流程

7.3.1 流程图



7.3.2 流程说明

1. Host/Service Tool 向 GSP 下发 SoC Key 轮换请求。
2. GSP 请求 eHSM 对本次 SoC Key 写操作进行鉴权。

3. 鉴权失败时，不打开 efuse 写权限，不执行任何 Key Slot 或 Bitmap 更新。
4. 鉴权通过后，eHSM 设置临时授权通过标志，并打开 eHSM Core 写 efuse 的逻辑。
5. 将新的 SoC FW Verify Key、SoC FW Encrypt Key 或 SoC Debug Key 写入新的 efuse Key Slot。
6. 设置新 Key Slot 的 Life Cycle 为可用，并完成 Key Type、Slot 和 CRC 检查。
7. 新 Key Slot 写入和校验完成后，更新对应的 Active Bitmap，使新 Slot 在下一次启动时生效。
8. 操作完成后立即关闭 efuse 写逻辑，并清除临时授权标志。
9. 复位后，eHSM ROM 重新读取 Active Bitmap，并选择新的验签、解密和 Debug Key Slot。
10. BootROM 使用新的 Active Slot 验证和解密 FMC。
11. Host/Service Tool 向 GSP/C908 下发 SoC Key 轮换请求。
12. GSP 请求 eHSM 对本次 SoC Key 写操作进行鉴权。
13. 鉴权失败时，不打开 OTP 写权限，不执行任何 Key Slot 或 Bitmap 更新。
14. 鉴权通过后，eHSM 设置临时授权通过标志，并打开 eHSM Core OTP 写逻辑。
15. 将新的 SoC FW Verify Key、SoC FW Encrypt Key 或 SoC Debug Key 写入新的 OTP Key Slot。
16. 设置新 Key Slot 的 Life Cycle 为可用，并完成 Key Type、Slot 和 CRC 检查。
17. 新 Key Slot 写入和校验完成后，更新对应的 Active Bitmap，使新 Slot 在下一次启动时生效。
18. 操作完成后立即关闭 OTP 写逻辑，并清除临时授权标志。
19. 复位后，eHSM ROM 重新读取 Active Bitmap，并选择新的验签、解密和 Debug Key Slot。
20. BootROM 使用新的 Active Slot 验证和解密 FMC。

8. 板级/OOB 安全边界

本章将 OOB MCU / 板级安全 MCU 纳入板级安全边界。BMC/OOB Host 仍按外部不可信请求方处理；OOB MCU 是板级安全控制点，可以承载带外管理、电源/复位、JTAG MUX 控制和 SPI 刷写 FMC，但不替代 eHSM Root of Trust，不持有根密钥，不直接放行任何 SoC 固件。

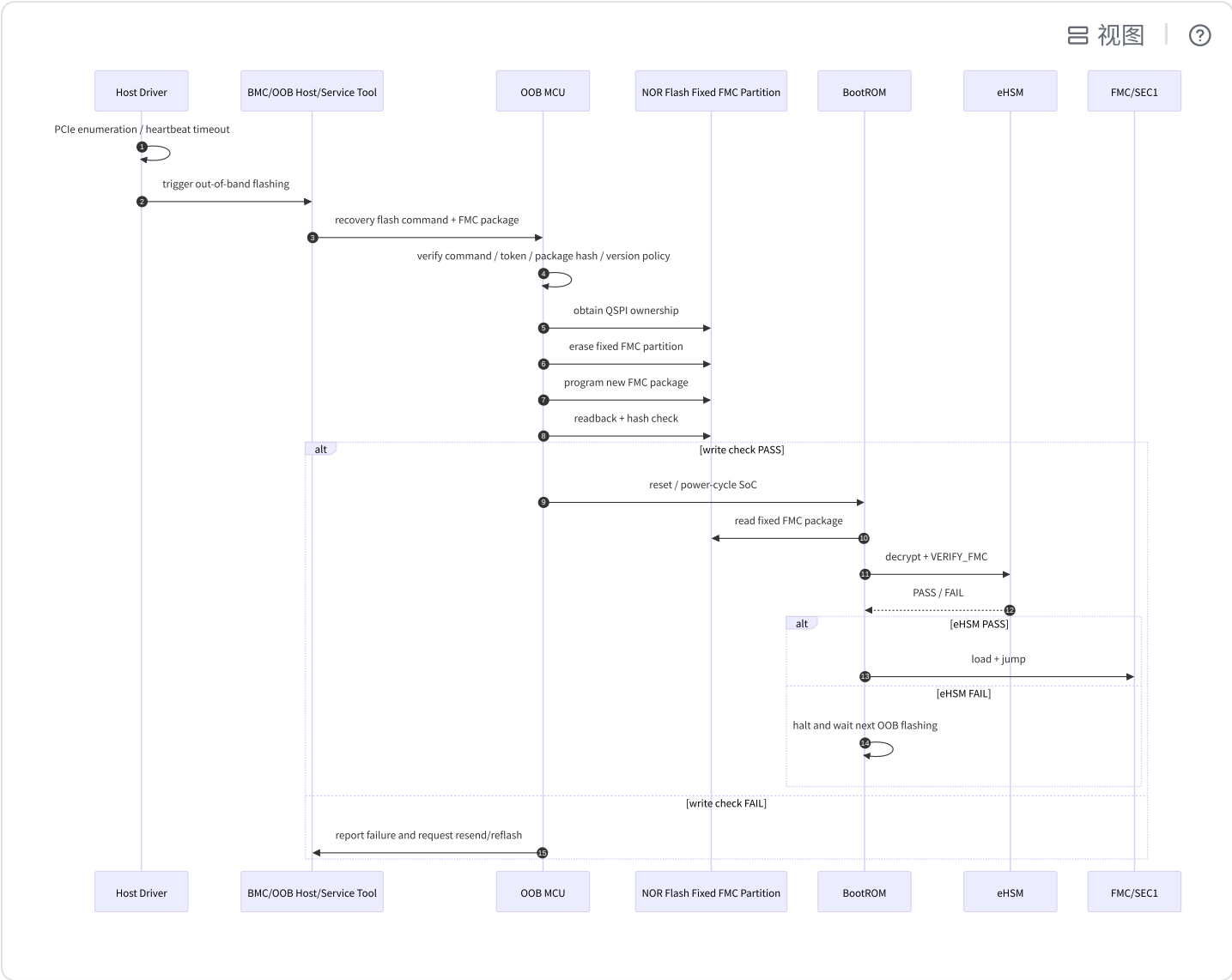
8.1 板级/OOB 安全架构

1. BMC/OOB Host 向 OOB MCU 发送管理、Debug、电源或刷机请求。
2. OOB MCU 在板级安全域内执行独立 secure boot 和授权检查。
3. 刷机时，OOB MCU 在可信环境内直接写 NOR Flash 固定 FMC 分区。
4. SoC 正常运行时，OOB MCU 可以把受控管理请求转发给 GSP。
5. OOB MCU 不访问 eHSM Root Key，不替代 eHSM 启动裁决。

8.2 OOB MCU 通过 SPI 刷写 FMC

OOB MCU 刷机流程是独立于 SoC 正常运行态更新的带外流程。触发条件是 Host/BMC 发现卡无法连接、PCIe 枚举失败、心跳超时或运维/制造授权刷机。刷机过程不经过 SoC；SoC 只在下一次启动时验证刷入的 FMC。

OOB MCU SPI 刷写与 SoC 启动验证关系图：



图下步骤说明：

1. Host 检测 PCIe 枚举失败或心跳超时。
2. Host/BMC/OOB Host 向 OOB MCU 下发 recovery flash command 和 FMC package。
3. OOB MCU 校验命令、token、包 hash 和版本策略。
4. OOB MCU 通过 SPI 擦除、写入并读回校验固定 FMC 分区。
5. 复位 SoC 后，BootROM/eHSM 对新 FMC 执行 verify + decrypt，决定是否跳转。

9. 多 Die 启动与证明策略

本章定义 NGU800P 双 Die 场景下的启动、固件搬运、Firewall 隔离、Debug 管控和远程证明策略。

9.1 基本原则

1. Die0 执行完整安全启动，Die0 eHSM 提供完整 SoC Root of Trust 和安全服务。
2. Die1 不独立建立新的 SoC Root of Trust，不独立执行镜像验签和解密。
3. Die1 启动镜像由 Die0 eHSM 执行验签、解密、反回滚和策略检查。
4. Die1 镜像在 Die0 验签解密后，通过 UCle 点对点传输到 Die1 受 Firewall 保护的 SRAM。
5. Die1 固件 measurement、远程证明和 LCS / Debug 权限管控由 Die0 统一纳入。
6. Die1 默认所有 Debug 接口关闭。
7. Die1 的 Debug 接口只能由 Die0 在鉴权通过后
8. 。
9. Die1 Firewall 配置由 Die1 管理子系统启动核 C908 执行，配置权限受 Die0/eHSM/LCS 策略约束。
10. Die1 固件明文在 UCle 和 Die1 SRAM 中只允许 Die0 受控管理通路、Die1 启动核和目标核访问

9.2 Die1 启动流程

Die1 本地启动动作在安全路径和非安全 bring-up 路径下基本一致，差异主要体现在 Die0 是否对 Die1 镜像执行 eHSM 验签、解密、反回滚和 release 策略检查。

9.3 Die1 Debug 管控

Die1 Debug 管控采用默认关闭、鉴权打开的策略。

1. Die1 默认所有 Debug 接口关闭。
2. Die1 本地不得自行打开 Debug 接口。
3. Debug 打开请求必须由 Die0 发起。
4. Die0 侧必须先完成 Debug 鉴权。
5. 鉴权通过后，Die0 通过 UCIE 配置 Die1 Debug gating。
6. 非User阶段，Die1 Debug由Die0初始化时打开

9.4 Die1 Measurement 策略

由于Die1的所有固件与Die0相同，所以不单独记录Die1的固件状态。